

Algorithms Problem Solving

Level 2

Fill Array With Keys



Algorithms Problem Solving

Level 2

Fill Array With Keys

Definition

- ◆ This program asks the user how many keys to generate.
- ◆ It generates random keys and stores them in an array of strings.
- ◆ Each key contains four groups of four uppercase letters separated by ` - `.
- ◆ Finally, it prints all generated keys.

Explanation

- ◆ ``GetRandomCharacter()`` generates a random character according to the selected character type.
- ◆ ``GenerateWord()`` generates a word with a specific length using random characters.
- ◆ ``GenerateKey()`` uses ``GenerateWord()`` four times to create one key.
- ◆ Each group contains ``4`` uppercase letters.
- ◆ The four groups are separated by ``-``.
- ◆ Example:
 CRTW-AWPH-LUEU-UMPO
- ◆ ``FillArrayByKeys()`` generates one key for each position in the array and stores it.
- ◆ ``PrintArrayElements()`` prints every key together with its array index.

Steps in main()

1. Ask the user how many keys to generate.
2. Create an array of strings.
3. Call `FillArrayByKeys()`.
4. Generate one key using `GenerateKey()`.
5. Store the generated key in the array.
6. Repeat until the requested number of keys is generated.
7. Print all keys with their array indexes.

💡 Example

- ◆ Number of keys: `5`
- ◆ `GenerateKey()` creates keys with this structure:
XXXX-XXXX-XXXX-XXXX
- ◆ One possible result:
Array[0]: CRTW-AWPH-LUEU-UMPO
Array[1]: AQUE-EKVG-UATH-FBTT
Array[2]: HUTF-VWSI-HMVF-DXIV
Array[3]: QBSO-WMIV-KHQL-MKZO
Array[4]: PHIW-UKSF-SWGU-CWOB
- ◆ Every run can produce different keys because the characters are generated randomly.

➡ Result

```
Array[0]: CRTW-AWPH-LUEU-UMPO  
Array[1]: AQUE-EKVG-UATH-FBTT  
Array[2]: HUTF-VWSI-HMVF-DXIV  
Array[3]: QBSO-WMIV-KHQL-MKZO  
Array[4]: PHIW-UKSF-SWGU-CWOB
```

📌 Note

- ◆ The key is built using:
`GenerateWord(enCharType::CapitalLetter, 4)`
- ◆ This generates four uppercase letters.
- ◆ `GenerateKey()` repeats this four times:
`XXXX-XXXX-XXXX-XXXX`
- ◆ The `string arr[100]` array stores each complete key as one string.
- ◆ The function hierarchy is:

`FillArrayByKeys()`



`GenerateKey()`



`GenerateWord()`



`GetRandomCharacter()`



`RandomNumber()`

- ◆ This is an example of breaking a larger task into smaller reusable functions.

⊘ Warning

- ◆ The array has a maximum size of `100`, so the number of keys should not exceed `100`.
- ◆ `ReadArrayElements()` is defined but not used.
- ◆ `cmath` and `corecrt\_math\_defines.h` are unnecessary.
- ◆ `SmallLetter`, `SpecialChar`, and `Digit` are defined in the enum but are not used by `GenerateKey()`.
- ◆ `GetRandomCharacter()` does not have a `default` case, so an unexpected enum value would not return a defined character.
- ◆ `rand()` is pseudo-random and is not suitable for security-sensitive keys.
- ◆ The generated keys are random-looking strings, not cryptographically secure keys.



Comparison

▨ `GenerateWord()`

- ✓ Generates a sequence of random characters.
- ✓ Receives the character type and length.
- ✓ Example: `GenerateWord(CapitalLetter, 4)` → `CRTW`.

▤ `GenerateKey()`

- ✓ Combines four generated words into one key.
- ✓ Adds `-` between the groups.
- ✓ Example: `CRTW` + `AWPH` + `LUEU` + `UMPO` → `CRTW-AWPH-LUEU-UMPO`.

✦ Summary

- ✓ Ask the user how many keys to generate.
- ✓ Generate each key using four groups of four uppercase letters.
- ✓ Store each key as a `string` in the array.
- ✓ Use `FillArrayByKeys()` to fill the array.
- ✓ Use `PrintArrayElements()` to display all generated keys.
- ✓ The final key format is `XXXX-XXXX-XXXX-XXXX`.

Thank you 😊